

VAJA 6

STATIČNI IN DINAMIČNI PREGLED KODE

Ekosistem simulacija: Lisica – Zajec – Radič + Pregled Eufficient

1 UVOD

Vaja 6 nadgrajuje Vajo 5, v kateri je bila izvedena sistemska refaktoracija ekosistem simulacije (lisica – zajec – radič) v programskem jeziku Python s knjižnico pygame. Namen te vaje je izvedba formalnega pregleda kakovosti kode v vlogi QA (Quality Assurance) inženirja.

Pregled obsega dva komplementarna pristopa – statični in dinamični – ter dodatno analizo spletne platforme Eufficient.

- **Statični pregled** zahteva branje kode brez izvajanja in iskanje napak, ozkih grl ter možnih izboljšav.
- **Dinamični pregled** z merjenjem časa posameznih funkcij potrdi ali ovrže napovedi statične analize.

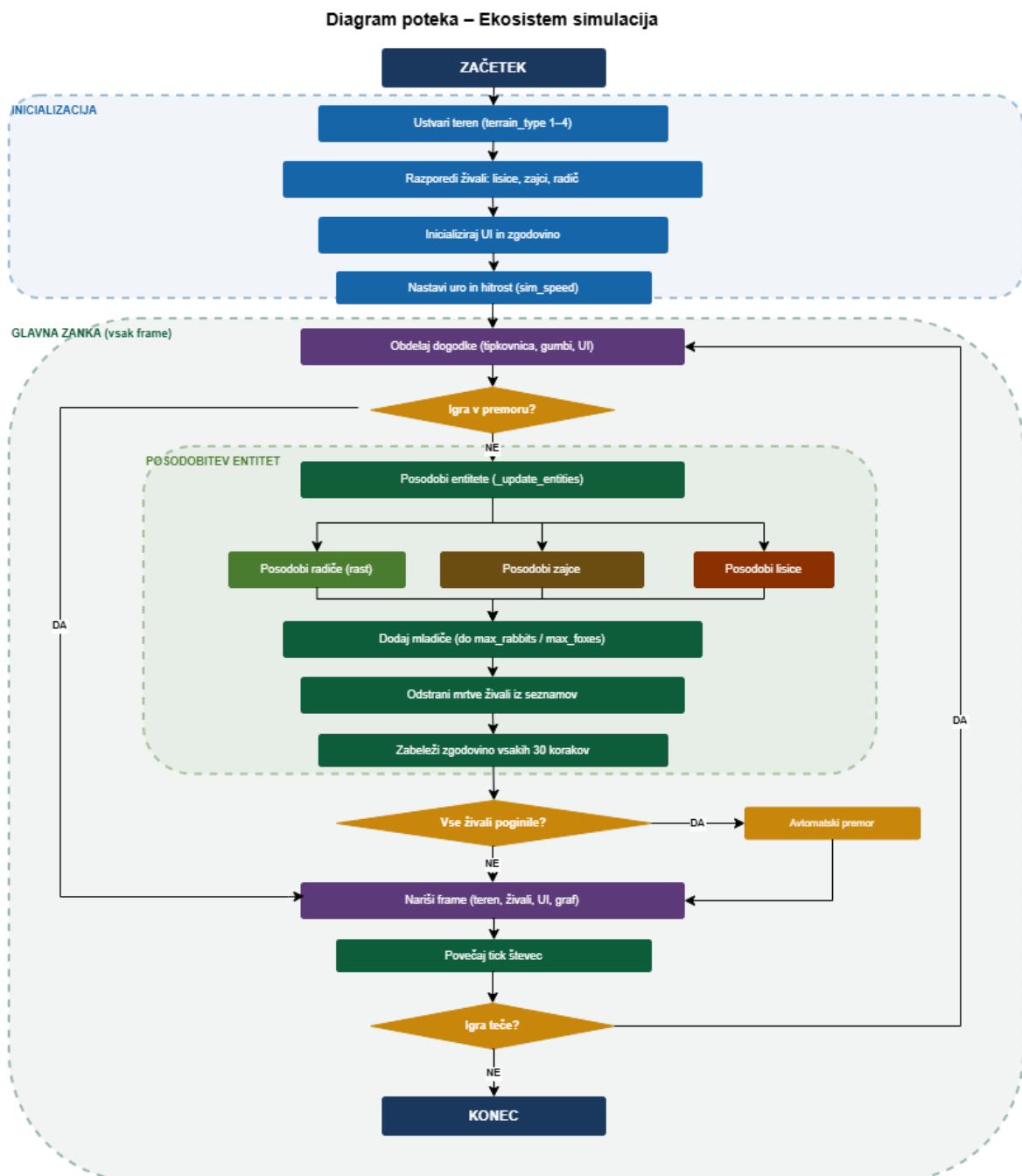
Struktura poročila:

- Diagram poteka – postopek pregleda
- Statični pregled – napake, ozka grla
- Dinamični pregled – meritve cProfile in ročne meritve
- Primerjava statično vs dinamično
- Pregled Eufficient
- Zaključek

2 DIAGRAM POTEKA

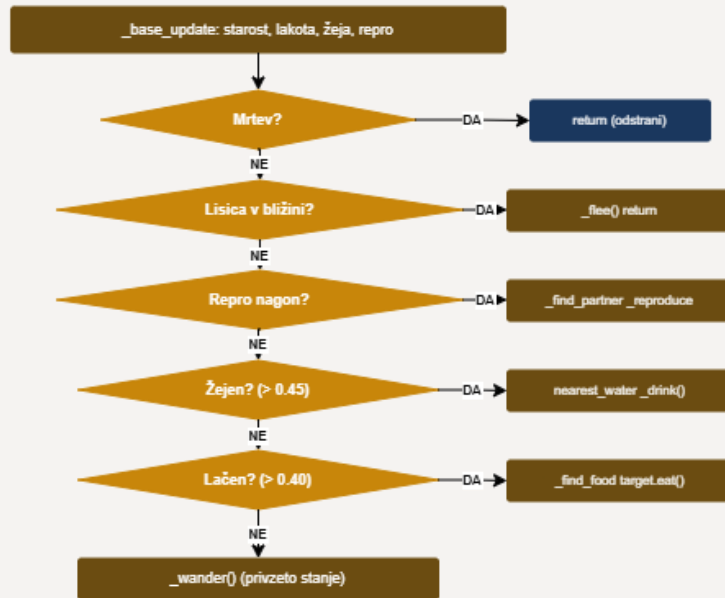
Diagram poteka za vajo 6 prikazuje celoten postopek pregleda kode – od statičnega pregleda petih datotek, prek dinamičnega merjenja zmogljivosti, do analize spletne strani Eufficient in pisanja poročila.

Narisan je bil v orodju **draw.io**, ki je brezplačno spletno orodje za risanje diagramov.



RABBIT.update()

Logika zajca (update)



FOX.update()

Logika lisice (update)

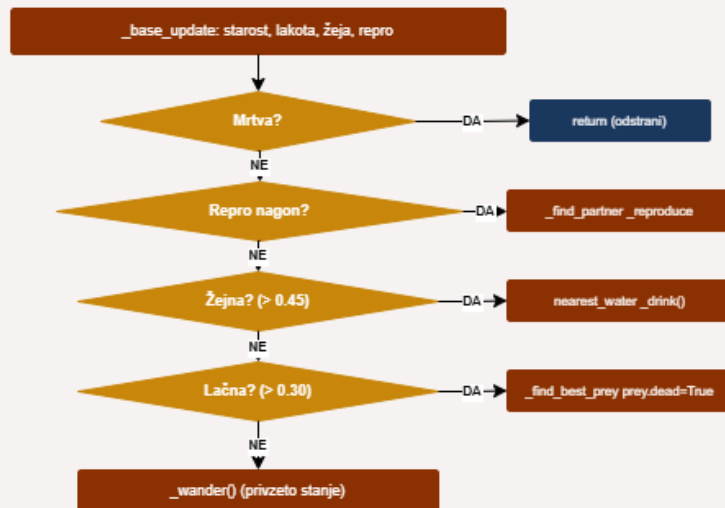
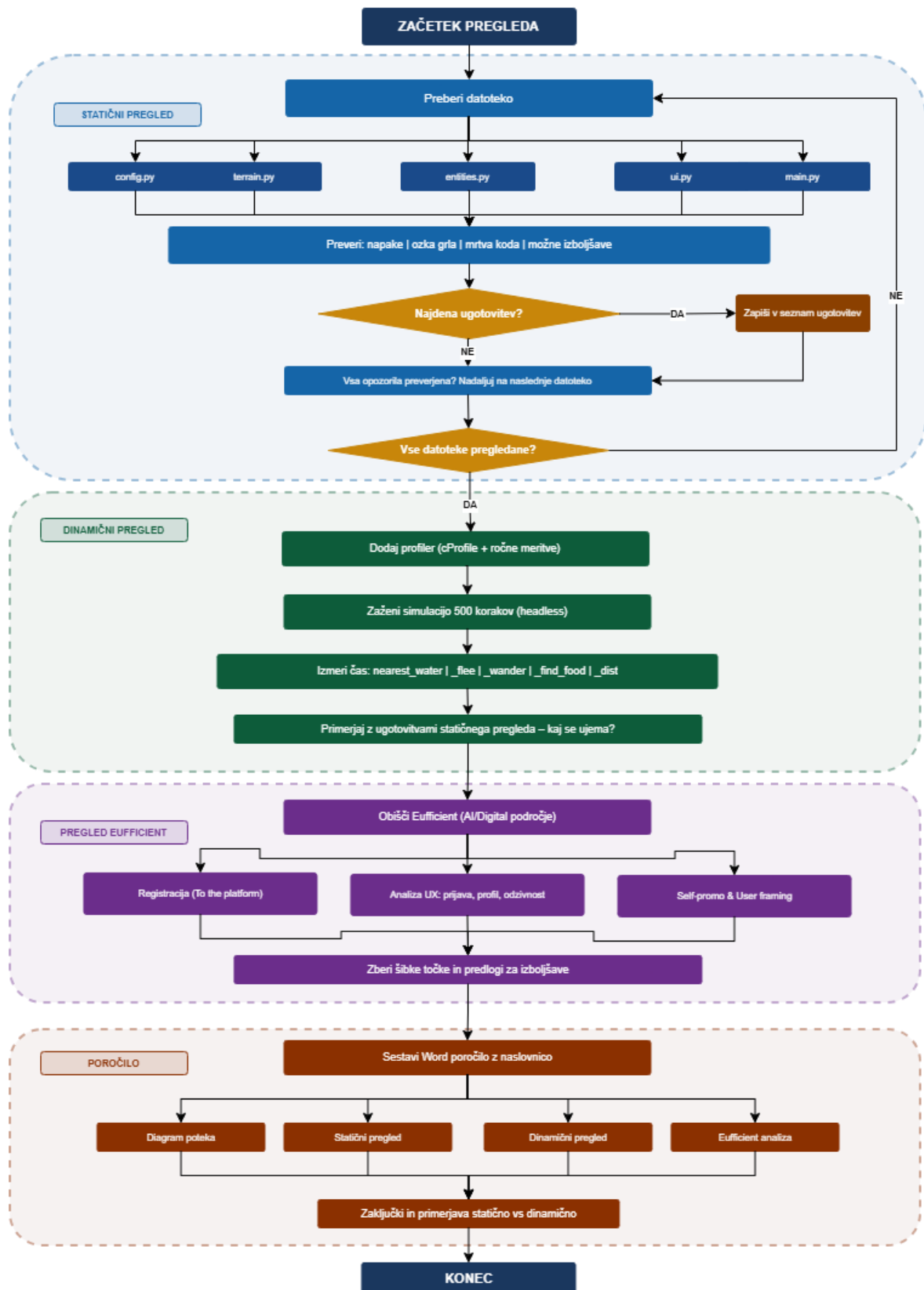


Diagram poteka – Pregled kode (Vaja 6)



3 STATIČNI PREGLED KODE

Statični pregled je bil izveden z branjem vseh petih datotek brez izvajanja kode. Iskali smo dve kategorije težav (mrtve kode ni bilo):

- ozka grla (počasna koda) in
- logične napake.

Poleg tega so bile zabeležene možne izboljšave.

3.1 OZKA GRLA (PERFORMANCE)

nearest_water() – kvadratna zanka [terrain.py, vrstica 307]

Kaj dela: Ko je žival žejna, pokliče metodo nearest_water(), ki poišče najbližjo celico z vodo. Metoda pregleda vse celice v kvadratu okoli živali v razdalji max_dist pikslov in vrne položaj najbližje vodne celice.

Kaj je problem: Metoda preišče vsako celico v kvadratu, ne glede na to, ali je voda ali kopno. Za zajca z vrednostjo max_dist = 320 px in velikostjo celice cell = 8 px je search_r = 41, kar pomeni kvadrat velikosti $83 \times 83 = 6.889$ celic na vsak klic.

```
# Preiščemo kvadrat okoli (r0, c0) z radijem search_r in poiščemo najbližjo celico z vodo.
for dr in range(-search_r, search_r + 1):
    for dc in range(-search_r, search_r + 1):
        r = r0 + dr
        c = c0 + dc

        # Preverimo, ali je (r, c) znotraj meja mreže in ali je voda.
        if 0 <= r < self.rows and 0 <= c < self.cols:

            # Če je celica voda, izračunamo njeno središče in oddaljenost od (px, py).
            if self.grid[r, c] == WATER:
```

Časovna zahtevnost je $O(\text{search_r}^2)$. Pri 100 žejnih živalih hkrati to znese 688.900 pregledov na vsak frame – samo za iskanje vode.

Rešitev: Namesto da metoda vsakič znova pregleda celotno mrežo, bi ob inicializaciji terena enkrat sestavili seznam vseh vodnih celic. Ko žival išče vodo, pregledamo samo ta seznam, ki vsebuje le dejanske vodne celice in ne vseh 6.889 celic v kvadratu. S tem bi se izognili ponavljajočemu pregledovanju praznih celic in bistveno zmanjšali število operacij na frame.

`_find_food()` – $O(n)$ za vsakega zajca [entities.py, vrstica 354]

Kaj dela: Ko je zajec lačen, pokliče metodo `_find_food()`, ki pregleda seznam vseh radičev in poišče najbližjega, ki še ni bil pojeden.

Kaj je problem: Metoda vsakič pregleda celoten seznam radičev, vključno s tistimi, ki so že pojedeni. Pojeden radič nikoli ne izgine iz seznama clovers, zato pri 500 radičih, od katerih je 400 pojedenih, vsak zajec preveri vseh 500 elementov – 80 % dela je že od začetka nepotrebno.

```
# Poišče najbližji nepojeden radič znotraj zaznavnega radija.
def _find_food(self, clovers):
    new = []
    best, best_score = None, self.sense_radius

    # Preišče vse radiče in izbere tistega, ki ni pojeden in je najbližje
    # (manjša razdalja = boljši rezultat).
    for clover in clovers:
        if clover.eaten:
            continue
        distance = _dist(self.x, self.y, clover.x, clover.y) # razdalja do radiča
        if distance < best_score: # bližje = boljše
            best_score, best = distance, clover
    return best
```

Rešitev: Pojedene radiče bi odstranili iz seznama takoj, ko so pojeni, ali pa bi vzdrževali ločen seznam samo aktivnih radičev. Tako bi metoda pregledovala le dejanske radiče in se izognila nepotrebni delu.

`list.remove()` – $O(n)$ x k odstranitv [main.py, vrstica 166]

Kaj dela: Med posodabljanjem entitet metoda `_update_entities()` iz seznama odstrani vse živali, ki so med izvajanjem umrle, z uporabo klica `list.remove()`.

Kaj je problem: `list.remove()` išče element od začetka seznama vse do tistega mesta, kjer ga najde – kar je operacija $O(n)$. Pri 300 zajcih in 20 smrtih na frame to pomeni povprečno $20 \times 150 = 3.000$ primerjav samo za brisanje.

```
for rabbit in self.rabbits[:]:
    rabbit.update(dt, terrain, self.foxes, self.rabbits, self.clovers, new_rabbits) # Posodobi zajca

    if rabbit.dead: # Če je zajec mrtev, ga odstrani iz seznama
        self.rabbits.remove(rabbit)
```

Rešitev: Namesto `list.remove()` bi seznam prefiltrirali z izpeljanim seznamom, ki v enem prehodu ohrani samo žive živali in se izogne ponavljajočemu iskanju.

Pojeden radič ostane v seznamu [entities.py, vrstica 72]

Kaj dela: Ko zajec doseže radič, pokliče `target.eat()`, ki radič označi z zastavico `self.eaten = True`. Radič ostane viden na mapi kot temnejša pikica.

Kaj je problem: Pojeden radič nikoli ne izgine iz seznama `clovers`. Vsak zajec, ki išče hrano, mora zato pregledati celoten seznam – vključno z vsemi pojedenimi radiči – in pri vsakem posebej preveriti zastavico `eaten`.

```
# Označi radič kot pojeden
def eat(self): new *
    self.eaten = True
```

```
# Poišče najbližji nepojeden radič znotraj zaznavnega radija.
def _find_food(self, clovers): new *
    best, best_score = None, self.sense_radius

    # Preišče vse radiče in izbere tistega, ki ni pojeden in je najbližje
    # (manjša razdalja = boljši rezultat).
    for clover in clovers:
        if clover.eaten:
            continue
```

Pri 500 radičih, od katerih je 400 pojedenih, vsak zajec pregleda vseh 500 in 80 % dela je nepotrebnega.

Rešitev: Namesto zastavice bi pojeden radič takoj odstranili iz seznama `clovers`. `_find_food()` bi tako pregledovala samo aktivne radiče brez kakršnega koli preverjanja.

Kopiranje seznamov vsak frame [main.py, vrstica 162]

Kaj dela: Med posodabljanjem entitet metoda `_update_entities()` ustvari kopijo celotnega seznama živali z rezanjem `[:]`, da se med iteracijo izogne napakam pri hkratnem brisanju elementov.

```
for rabbit in self.rabbits[:]:
```

```
    for fox in self.foxes[:]:
```

```
        for clover in self.clovers[:]:
```

Kaj je problem: `self.rabbits[:]` ustvari popolno kopijo celotnega seznama vsakič, ko se frame osveži – torej 60-krat na sekundo. Pri 300 zajcih in 50 lisicah to pomeni 350 novih objektov v pomnilniku na vsak frame, čeprav je kopija potrebna zgolj kot varnostni ukrep med iteracijo.

Rešitev: Namesto kopiranja bi mrtve živali označili in jih odstranili šele po končani iteraciji z izpeljanim seznamom. S tem bi se izognili ustvarjanju kopij in pomnilnik ne bi bil po nepotrebnem obremenjen.

3.2 LOGICNE NAPAKE

`_inherit()` vedno uporablja `fox_variation` [entities.py, vrstica 40]

Kaj dela: Funkcija `_inherit()` izračuna podedovano lastnost mladiča kot povprečje vrednosti obeh staršev, ji doda naključno variacijo in mutacijo ter vrne končno vrednost.

Kaj je problem: Funkcija pri izračunu variacije vedno uporabi `cfg.fox_variation`, tudi kadar se razmnožujejo zajci. To je konceptualna napaka – zajčji mladič podeduje lastnosti z variacijo, ki je namenjena lisicam.

```
# Lastnost potomca = povprečje staršev, nato variacija in možna mutacija.
def _inherit(v_a: float, v_b: float, cfg) -> float: 12 usages new *
    base = (v_a + v_b) / 2.0
    base = _vary(base, cfg.fox_variation)
    base = _mutate(base, cfg)
    return base
```

V praksi se napaka numerično ne pozna, ker sta vrednosti trenutno enaki (`cfg.fox_variation = cfg.rabbit_variation = 0.10`). Če bi ju v prihodnje spremenili na različni vrednosti, bi zajci napačno dedovali lastnosti.

Rešitev: Funkciji bi dodali parameter `variation`, ki ga klic posreduje glede na vrsto živali. Tako bi vsaka vrsta uporabljala svojo pravilno vrednost (klic posebej za zajce in lisice).

`self.age` se ne množi s `sim_speed` [entities.py, vrstica 131]

Kaj dela: Metoda `_base_update()` vsak frame posodablja osnovne življenjske parametre živali – starost, žejo in lakoto – glede na pretečen čas `dt` in hitrost simulacije `sim_speed`.

Kaj je problem: Žeja in lakota se pravilno množita s `sim_speed`, starost pa ne. To pomeni, da pri `sim_speed = 5` žival umre od lakote 5× hitreje, od starosti pa enako hitro kot pri `sim_speed = 1` kar je nedosledno.

```
self.age += dt      self.thirst += dt * self._thirst_rate * self.cfg.sim_speed * thirst_mult
self.hunger += dt * self._hunger_rate * self.cfg.sim_speed
```

V praksi to pomeni, da pri visoki hitrosti simulacije živali umirajo od lakote in žeje, preden se sploh postarajo – starost kot vzrok smrti praktično izgine.

Rešitev: Starost bi množili s `sim_speed` enako kot ostale parametre, da bi bilo obnašanje vseh življenjskih parametrov dosledno.

`_wander_timer` se ne množi s `sim_speed` [entities.py, vrstica 201]

Kaj dela: `_wander_timer` določa, kako pogosto žival menja smer tavanja. Ko timer pade na nič, žival izbere novo naključno smer in timer se ponastavi.

Kaj je problem: `_wander_timer` se zmanjšuje brez `sim_speed`, medtem ko se vsi drugi timerji množijo z njim. To pomeni, da žival menja smer enako pogosto v realnem času ne glede na hitrost simulacije, a ker se giblje hitreje, v tem času prehodi večjo razdaljo – kar povzroči nelogično tavanje pri višjih hitrostih.

```
self._wander_timer -= dt                # brez sim_speed
# vsi drugi timerji:
self.repro_cooldown = max(0.0, self.repro_cooldown - dt * self.cfg.sim_speed)
self.newborn_timer  = max(0.0, self.newborn_timer  - dt * self.cfg.sim_speed)
```

Predstavljajmo si zajca, ki pri `sim_speed = 1` naredi 3 korake, nato menja smer. Pri `sim_speed = 5` naredi 15 korakov v isti smeri – ker timer teče enako hitro, žival pa se giblje 5× hitreje in šele nato menja smer. Tavanje postane čedalje bolj ravnočrtno in nenaravno.

Rešitev: `_wander_timer` bi množili s `sim_speed` enako kot vse ostale timerje, da bi žival pri vsaki hitrosti simulacije menjavala smer enako pogosto v simulacijskem času.

`_flee()` – fox not in nearby je $O(n)$ [entities.py, vrstica 419]

Kaj dela: Med begom zajec preveri, ali se katera lisica nahaja preblizu potencialne smeri umika. Pogoj fox not in nearby zagotovi, da pri tem ne upošteva lisic, ki so že preblizu in se jim ne more izogniti.

Kaj je problem: fox not in nearby deluje kot iskanje po seznamu – Python mora za vsako lisico **enega po enega** pregledati celoten seznam nearby, da ugotovi, ali je že v njem. To je operacija $O(n)$ za vsako lisico posebej.

```
if any(_dist(tx, ty, fox.x, fox.y) < 30 for fox in foxes if fox not in nearby):
    ndx, ndy = -ndx, ndx
```

Pri 20 lisicah in 10 elementih v nearby to pomeni do **200 primerjav** samo za en klic `_flee()` – in ta se izvaja za vsakega zajca na vsak frame.

Rešitev: Seznam nearby bi pretvorili v množico (set), pri kateri je preverjanje prisotnosti elementa $O(1)$ namesto $O(n)$, saj množica ne išče po vrstnem redu ampak direktno.

4 DINAMICNI PREGLED

Dinamični pregled je bil izveden s pomočjo Python modula cProfile in ročnih meritev z modulom time.perf_counter().

Simulacija je tekla 500 korakov v načinu brez zaslona (headless pygame).

Populacija ob zagonu: 8 lisic, 30 zajcev, 80 radičev.

4.1 CPROFILE – NAJPOGOSTEJŠE IN NAJPOČASNEJŠE FUNKCIJE

cProfile meri skupni čas ki ga vsaka funkcija porabi med celotnim tekom simulacije. Rezultati pokažejo katere funkcije se najpogosteje kličejo in koliko časa skupno porabijo.

Mesto	Funkcija	Število klicev	Skupaj čas	% celote
1	Rabbit.update()	15.000	0.139 s	80 %
2	_wander()	18.287	0.066 s	38 %
3	_flee()	15.000	0.064 s	37 %
4	_dist()	124.989	0.034 s	20 %
5	is_passable()	19.098	0.029 s	17 %
6	_clamp()	19.000	0.026 s	15 %
7	Fox.update()	4.000	0.021 s	12 %
8	_base_update()	19.000	0.017 s	10 %
9	math.hypot()	125.702	0.013 s	8 %

1. Rabbit.update() – 15.000 klicev, 0.139s, 80% časa

To je glavna funkcija zajca, ki se pokliče vsak korak za vsakega zajca. $30 \text{ zajcev} \times 500 \text{ korakov} = 15.000 \text{ klicev}$. Porabi 80% celotnega časa simulacije ker v sebi pokliče vse ostale funkcije – beg, iskanje hrane, tavanje, pitje.

2. _wander() – 18.287 klicev, 0.066s, 38% časa

Tavanje je privzeto stanje živali – ko niso lačne, žejne ali prestrašene, samo tavajo. 18.287 klicev je celo več kot 15.000 (kolikor je klicev Rabbit.update), ker tavanje kličejo tudi lisice.

3. _flee() – 15.000 klicev, 0.064s, 37% časa

Vsak zajec na vsak korak preveri ali je lisica v bližini – tudi ko ni nobene lisice blizu. Funkcija se torej vedno pokliče, ne samo ko se zajec dejansko boji. To razloži visoko število klicev.

4. _dist() – 124.989 klicev, 0.034s, 20% časa

Izračun razdalje med dvema točkama se kliče povsod – pri begu, iskanju hrane, iskanju partnerja, lovu. Samo v 500 korakih skoraj 125.000 klicev. Funkcija sama je hitra, a se jo kliče tolikokrat da skupaj vzame 20% časa.

5. `is_passable()` – 19.098 klicev, 0.029s, 17% čas

Preverjanje ali je celica kopna se kliče vsakič ko se žival premakne – da ne stopi v vodo. 19.098 klicev ustreza skupnemu številu premikov vseh živali v simulaciji.

6. `_clamp()` – 19.000 klicev, 0.026s, 15% časa

Funkcija skrbi da žival ne zapusti meje mape. Pokliče se po vsakem premiku, zato ima podobno število klicev kot `is_passable()`. Sama po sebi ni počasna, a se kliče ob vsakem koraku vsake živali.

7. `Fox.update()` – 4.000 klicev, 0.021s, 12% časa

Enako kot `Rabbit.update()` – glavna funkcija lisice, ki se pokliče vsak korak za vsako lisico. $8 \text{ lisic} \times 500 \text{ korakov} = 4.000 \text{ klicev}$. Bistveno manj kot zajci, ker je lisic v simulaciji manj.

8. `_base_update()` – 19.000 klicev, 0.017s, 10% časa

Skupna funkcija za vse živali – posodablja starost, lakoto, žejo in reprodukcijski nagon. $19.000 \text{ klicev} = \text{skupaj } 30 \text{ zajcev} + 8 \text{ lisic} = 38 \text{ živali} \times 500 \text{ korakov}$. Pokliče se za vsako žival na vsak korak.

9. `math.hypot()` – 125.702 klicev, 0.013s, 8% časa

Vgrajena Python funkcija za izračun razdalje – kliče jo `_dist()`. Skoraj identično število klicev kot `_dist()` (124.989), kar potrди da `_dist()` skoraj vedno pokliče `math.hypot()`. Skupaj `_dist()` in `math.hypot()` predstavljata velik del obremenitve.

Na začetku simulacije živali niso žejne (`thirst = 0.0`). Funkcija `nearest_water()` se pokliče samo ko žeja preseže prag `thirst > 0.45`, zato se v prvih korakih simulacije sploh ne kliče. Ker cProfile sešteje čas čez vseh 500 korakov skupaj, funkcija ni bila poklicana dovolj pogosto da bi se uvrstila med najpočasnejše.

4.2 ROČNE MERITVE – ČAS NA EN KLIC

Ročne meritve so bile izvedene na podlagi ugotovitev statičnega pregleda. Vsaka funkcija, ki je bila v statičnem pregledu označena kot sumljiva, je bila izmerjena posebej poklicana 500 do 1000-krat zapored in izmerjen povprečen čas na en klic z modulom `time.perf_counter()`.

Poleg teh so bile izmerjene še funkcije na 1, 4 in 5 mestu, ki v statičnem pregledu niso bile posebej izpostavljene, a so bile dodane za primerjavo da vidimo kako hitre so glede na problematične funkcije in dobimo celotno sliko zmogljivosti simulacije.

Tako smo dobili točen čas enega klica, neodvisno od tega koliko krat se funkcija pokliče med celotno simulacijo.

Mesto	Funkcija	ms / klic	Opomba
1	<code>bake_surface()</code>	9.41 ms	Samo ob inicializaciji – ne vpliva na runtime
2	<code>nearest_water()</code>	0.95 ms	Najpočasnejša runtime funkcija – $O(\text{search_r}^2)$
3	<code>find_food()</code>	0.011 ms	Manj problematično kot napovedano statično
4	<code>find_best_prey()</code>	0.004 ms	Hitro
5	<code>find_partner()</code>	0.001 ms	Hitro
6	<code>flee()</code>	0.001 ms	Hitro per klic, a se kliče pri vsakem zajcu

`_bake_surface()` je z 9.41 ms daleč najpočasnejša funkcija, vendar se kliče samo enkrat ob inicializaciji terena – med izvajanjem simulacije ne vpliva na zmogljivost.

`nearest_water()` je z 0.95 ms najpočasnejša funkcija med izvajanjem. To potrjuje ugotovitev statičnega pregleda o kvadratni časovni zahtevnosti. cProfile je ni pokazal med problematičnimi ker se na začetku simulacije redko kliče – ročna meritev je razkrila resnično težavo.

`_find_food()` je bila v statičnem pregledu označena kot problematična, vendar je meritev pokazala samo 0.011 ms na klic – statični pregled je v tem primeru precenjeval težavo.

Ročne meritve so potrdile da **je `nearest_water()` največja težava v simulaciji**, hkrati pa so pokazale da nekatere statično napovedane težave v praksi niso tako problematične.

4.3 Ocena obremenitve na frame

Na podlagi ročnih meritev smo izračunali skupno obremenitev na en frame pri najslabšem scenariju – ko so vse živali hkrati žejne in se `nearest_water()` kliče za vsako žival posebej.

Izračun je preprost: čas enega klica pomnožimo s številom klicev na frame.

Funkcija	Klici/frame	ms/klic	Skupaj ms/frame
<code>nearest_water()</code>	~38	0.95 ms	36.2 ms
<code>_find_food()</code>	~30	0.011 ms	0.33 ms
<code>_find_partner()</code>	~38	0.001 ms	0.05 ms
<code>_find_best_prey()</code>	~8	0.004 ms	0.03 ms
SKUPAJ			~36.6 ms

Skupna obremenitev znaša **~36.6 ms na frame**, frame budget pri 60 fps pa je **16.7 ms**. To pomeni da simulacija samo za iskanje vode, hrane in partnerjev porabi **219% celotnega frame budgeta** – preden sploh nariše karkoli na zaslon.

Največji krivec je `nearest_water()` z 36.19 ms od skupnih 36.6 ms – torej praktično vsa obremenitev izvira iz ene same funkcije.

Opomba: Gre za oceno najslabšega primera. V praksi niso vse živali hkrati žejne, zato je dejanska obremenitev nižja. Kljub temu rezultat jasno pokaže da je `nearest_water()` kritično ozko grlo, ki bi ga bilo pri večji populaciji nujno optimizirati.

5 Primerjava statično vs dinamično

Primerjava ugotovitev statičnega in dinamičnega pregleda pokazuje katera statična opažanja so bila pravilna in kaj je dinamično testiranje dodatno odkrilo.

Ugotovitev	Statično	Dinamično	Ujemanje
nearest_water() je ozko grlo	Napovedano ($O(\text{search_r}^2)$)	Potrjeno: 0.95 ms/klic, 219% frame budgeta	DA
_flee() je problematičen	Napovedano ($O(n) + \text{any}()$)	Potrjeno: #3 v cProfile	DA
_find_food() bo počasen	Napovedano ($O(n)$ po radičih)	Delno: le 0.011 ms – manjši problem	DELNO
_wander() je počasen	Omenjena logična napaka	Odkrito: #2 v cProfile z 18.287 klici	PRESENEČENJE
_dist() se kliče pogosto	Omenjeno	Potrjeno: 124.989 klicev v 500 korakih	DA
nearest_water() ni v cProfile	Ni predvideno	Odkrito: ne pokaze se ker živali niso žejne na začetku	PRESENEČENJE

KLJUČNA UGOTOVITEV

Statični pregled je pravilno identificiral nearest_water() kot največjo težavo in _flee() kot pogosto klicano funkcijo. Dinamični pregled je to potrdil in dodal dve presenetljivi ugotovitvi:

- _wander() je #2 po skupnem času – ker je tavanje privzeto stanje večine živali.
- nearest_water() se ne pojavi v cProfile pri kratkem teku ker živali sprva niso žejne. Statična analiza je bila pravilna, a cProfile bi jo zamudil brez ročnih meritev.

To kaže da sta oba pristopa komplementarna – statični pregled identificira strukturne težave, dinamični pa razkriva dejanske obremenitve med izvajanjem.

6 Pregled spletne platforme EU.FFICIENT

EU.FFICIENT (eufficient.eu) je evropski projekt, financiran v okviru programa Horizon Europe, ki vzpostavlja skupnosti strokovnih facilitatorjev za izboljšanje procesov soustvarjanja po Evropi. Pregled je bil izveden z obiskom spletnega mesta in registracijskega obrazca na platformi community.eufficient.eu.

Hitri pregled strani in mnenje

Potrebna bi bila kakšna HERO naslovnica in tudi spodaj en osnovni tekst kjer bi se hitro in enostavno povedalo o čem se gre stran.

Tekst je premajhen je ga zelo težko brati.

Gumb »To the platform« mi prvo ne pove in ne razumem kaj je mišljeno da grem na platformo kakšno ni logično da te potem, da na obrazec.

Prvo bi moralo biti da se predstavi kaj je ta platforma in potem še le gumb povezava.

Prijava na novice je zelo čudno oblikovana lahko bi bila na sredini in tudi form bi lahko bil lepše pripravljen je dosti prostora med elementi.

Lahko bi socialne medije še kakšne dodali tik tok ali pa instagram.

Postopek registracije

Obrazec je razumen dosti večja pisava vidnejša vendar ima probleme:

- da ima veliko polji za izpolniti lahko koga odbije od tega,
- ni označbe obveznih polji nikjer napisano,
- spol ima 3 opcije (3. opcija raje nebi reko je zelo čudna opcija).
- Ni možnosti prijave preko Google ali pa kerega drugega
- Geslo nima indikatorja močnosti

Ko se prijavim in dobim pošto za sprejeti in ko kliknem povezavo me nazaj vrže na obrazec za registracijo namesto na prijavo.

Lahko bi že na domači strani bilo malo bolj opisano ali pa prikazano zakaj se ta platforma gre ker kot novi uporabnik kateri hitro prebere stran potem ne veš kako uporabljati kaj se dogaja.

Tudi ko si na portalu ne deluje estetsko dovršeno.

Responsive pregled (tablica in mobil)

Pri pregledu odzivnosti spletne strani je bilo ugotovljeno, da prikaz na tablicah in mobilnih napravah ni optimalen. Čeprav se elementi pri manjših zaslonih zložijo, je prilagoditev minimalna – besedilo je na mobilnih napravah manjše kot na namiznem računalniku, splošna izkušnja pa zaostaja za pričakovanji.

Glede na to, da velik delež uporabnikov danes spletne strani obiskuje prek telefona, bi bilo temu področju smiselno nameniti več pozornosti.

EU.FFICIENT

Welcome to EU.FFICIENT, a pioneering initiative poised to revolutionise co-creation practices across Europe. At the heart of our project lies a commitment to fostering vibrant Communities of Facilitators dedicated to advancing co-creation methodologies through targeted training, extensive networking, and immersive hands-on experiences.

EU.FFICIENT is designed to empower Expert Facilitators with tailored resources and a dedicated platform, facilitating seamless access to cutting-edge tools and methodologies.

OBJECTIVE

The main objective of EU.FFICIENT is to create, pilot and validate Communities of Expert Facilitators to improve co-creation processes in Europe, with a focus on personalised training, networking and hands-on learning.

[To the platform](#)

EU.FFICIENT PLATFORM

The EU.FFICIENT Platform is a dynamic hub designed to empower you through personalised training tailored to your sector, exclusive networking opportunities with industry leaders, and hands-on learning experiences that enhance your facilitator skills.

ABOUT

EU.FFICIENT is an innovative project aimed at enhancing collaborative processes across Europe through the establishment of Communities of Expert Facilitators (CEFs). These communities serve as platforms for industry experts, academia, public sector representatives, and society members to come together, share knowledge, and co-create solutions in various fields.

WHAT IS CO-CREATION?

By co-creation we refer to the collaborative process where academic researchers and industry/public sector professionals work together to develop new knowledge, technologies, products, and services. Its goal is to leverage the complementary strengths of academia and industry/public sector to create value and drive innovation. Co-creation involves the exchange of know-how, expertise, and resources between the parties, and requires a deep

[Read more ...](#)

WHAT ARE EXPERT FACILITATORS?

Expert Facilitators are individuals who possess the necessary skills and expertise to guide and facilitate the collaboration between the participating stakeholders, so-called Co-creators.

the necessary skills and expertise to guide and facilitate the collaboration between the participating stakeholders, so-called Co-creators.

Their roles include:

Institutional roles

- Higher education institutions (HEIs): Technology Transfer Officers, Business Liaison Officers, European/R&D Project Officers.
- Research and technology organisations (RTOs): Business Developers, Innovation Managers.

Intermediary roles

- Cluster Managers in innovation clusters.
- Innovation Managers in science and innovation parks.

Corporate and public sector roles

- Open Innovation Managers in large

institution's goals, helping you establish meaningful connections with potential industry partners and collaborators.

[Secure your spot today!](#)

PARTNERS

EU.FFICIENT aims at improving collaboration between industry, academia and public sector, and all three sectors are duly represented through consortium partners:

Katholieke Universiteit Leuven, UPC Technology Center, Karlsruhe Institute of Technology, i2m Consulting, Centre for Strategy & Evaluation Services Europe, Fondazione Politecnico di Milano, Politecnico di Milano, CentraleSupélec, University of Maribor, TICE, Flanders Make, EIT Manufacturing, SUNRISE Valley Science and Technology Park, University Industry Innovation Network, City of Skellefteå, Innopark

Name

First

Last

Entity

Role

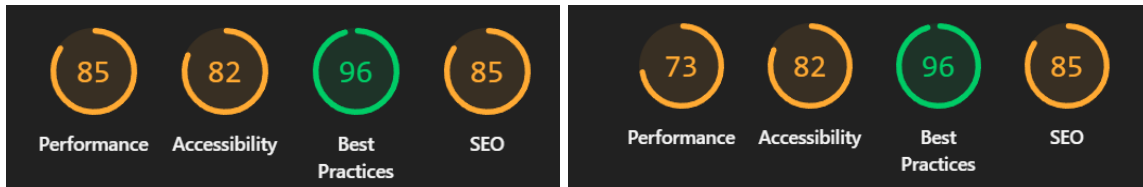
Country

Consent *

☐ If you wish to subscribe to our newsletter, at least your e-mail address is mandatory. All other information, as mentioned above.

Lighthouse

Spletna stran eufficient.eu je bila analizirana z orodjem Lighthouse za desktop in mobilno različico.



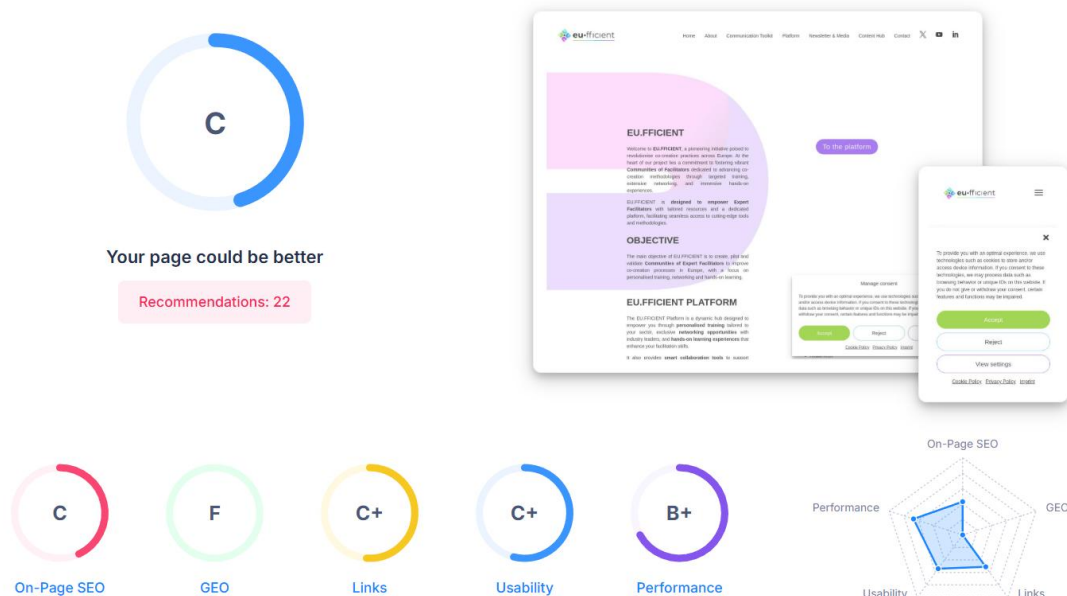
Najboljšo oceno je dosegla kategorija **Best Practices** z 96 točkami na obeh napravah, kar kaže na dobre tehnične temelje. **SEO** in **Accessibility** sta enaki na obeh – 85 in 82 točk.

Največja razlika je pri **Performance** – desktop doseže 85, mobilna različica pa samo **73 točk**. To potrjuje ugotovitev iz ročnega pregleda, da stran ni dobro optimizirana za mobilne naprave.

Vse kategorije razen Best Practices so v oranžnem območju, kar pomeni da stran deluje, a ima prostor za izboljšave – zlasti na mobilnih napravah

SEO REPORT

Spletna stran eufficient.eu je bila dodatno analizirana s SEO audit orodjem, ki je skupno ocenilo stran z **oceno C** in podalo **22 priporočil** za izboljšave.



Najslabšo oceno je prejela kategorija **GEO** z oceno F – manjkata Identity Schema in IImS.txt datoteka, kar vpliva na vidnost strani pri modernih iskalnikih in AI orodjih.

Med najpomembnejšimi priporočili so:

- **Manjka Meta Description** – visoka prioriteta, neposredno vpliva na prikaz v iskalnikih
- **Prekratek Title Tag** – srednja prioriteta
- **Premalo Header Tags** – srednja prioriteta, slabša struktura vsebine
- **Manjka Schema Markup** – srednja prioriteta, slabša razumljivost za iskalnike
- **Hitrost nalaganja** – srednja prioriteta, potrjuje ugotovitev iz Lighthouse analize

Večina priporočil (15 od 22) je nizke prioritete in zajema predvsem socialna omrežja in lokalni SEO, kar za evropsko platformo tega tipa ni kritično.

Ključni problem ostaja **On-Page SEO** – osnove kot so meta opis, naslov in struktura strani niso optimalno nastavljene.